

ACTIVIDAD 07: LAB: FILE PATH TRAVERSAL SIMPLE CASE

CNO IV (T37A): Seguridad Informática

Alumno: Alejandro García Morales
Profesor: Mtro. Servando López Contreras

Contenido

1. Nombre del Laboratorio	2
2. Nivel de Dificultad	2
3. Tipo de Explotación	2
4. Objetivo del Laboratorio.....	2
5. Marco Teórico y Fundamentación Técnica	2
5.1. Definición Formal de la Vulnerabilidad.....	2
5.2. Funcionamiento en el Sistema Operativo y Servidor Web	2
5.3. Análisis de Impacto en la Tríada CIA (Confidencialidad, Integridad y Disponibilidad)	3
5.4. Variantes Principales de Path Traversal	3
6. Procedimiento Paso a Paso y Uso Técnico de Burp Suite.....	4
Paso 1: Acceso e inicialización del entorno del laboratorio.....	4
Paso 2: Configuración del entorno de auditoría e inicio del navegador.....	4
Paso 3: Carga y mapeo visual de la aplicación objetivo	5
Paso 4: Inspección de peticiones HTTP y selección del vector en HTTP History	6
Paso 5: Aislamiento del parámetro vulnerable en el módulo Repeater	7
Paso 6: Inyección del Payload y exfiltración de /etc/passwd.....	8
Paso 7: Validación y confirmación del reto resuelto.....	9
7. Registro de Evidencias Técnicas	10
8. Explicación Técnica del Payload	11
Payload Utilizado:.....	11
Análisis de la Explotación:	11
9. Mitigación Recomendada y Principios de Defensa en Profundidad	11
9.1. Eliminar la Entrada Directa del Usuario en Operaciones de Ficheros	11
9.2. Verificación de Rutas Mediante Canonicalización (Path Canonicalization)	12
9.3. Principio de Menor Privilegio y Enjaulamiento (Hardening del Entorno) ..	12
10. Referencias bibliográficas.....	12

1. Nombre del Laboratorio

Lab: File path traversal, simple case

Plataforma: PortSwigger Web Security Academy

Enlace al laboratorio: <https://portswigger.net/web-security/file-path-traversal/lab-simple>

2. Nivel de Dificultad

Apprentice

3. Tipo de Explotación

- **Categoría principal:** Inyección de secuencias de escape de directorio (File Path Traversal).
- **Mecanismo:** Manipulación de parámetros de entrada no sanitizados en solicitudes HTTP de lectura de archivos locales.

4. Objetivo del Laboratorio

Identificar la falta de sanitización en el parámetro que gestiona la carga y despliegue de imágenes de productos en el servidor web, y explotar dicha vulnerabilidad para evadir la restricción del directorio base mediante secuencias de escape relativas (../), extrayendo exitosamente el contenido del archivo sensible del sistema operativo /etc/passwd.

5. Marco Teórico y Fundamentación Técnica

5.1. Definición Formal de la Vulnerabilidad

La vulnerabilidad de File Path Traversal, clasificada en el estándar CWE-22: Improper Limitation of a Pathname to a Restricted Directory ocurre cuando una aplicación web utiliza entradas suministradas por el usuario para construir rutas de acceso a archivos en el sistema de almacenamiento subyacente sin validar, sanitizar o filtrar adecuadamente caracteres especiales de navegación jerárquica.

En el contexto del estándar OWASP Top 10:2021, esta falla se agrupa dentro de la categoría A01:2021 - Broken Access Control (Control de Acceso Roto), dado que el sistema no hace cumplir la política de aislamiento de recursos, permitiendo que un usuario no autenticado acceda a información restringida del servidor a la cual no debería tener visibilidad.

5.2. Funcionamiento en el Sistema Operativo y Servidor Web

Los sistemas operativos de arquitectura Unix/Linux estructuran su sistema de ficheros de forma jerárquica a partir de un único directorio raíz denominado /. Para hacer referencia a elementos dentro de este árbol, se emplean dos notaciones:

1. **Rutas absolutas:** Indican la localización exacta desde la raíz (por ejemplo, /var/www/images/producto.jpg).

2. **Rutas relativas:** Indican la posición respecto al directorio de trabajo actual. En los estándares POSIX:
 - El carácter punto simple (.) representa el directorio actual.
 - El carácter punto doble (..) representa el directorio padre (un nivel superior).

Cuando una aplicación web recibe una petición dinámica (por ejemplo, GET /image?filename=producto.jpg), el código del backend frecuentemente concatena la entrada del usuario a un prefijo estático:

```
$$\text{Ruta Final} = \text{BASE\_DIRECTORY} + \text{userInput}$$
```

Si la variable BASE_DIRECTORY está fijada en /var/www/images/ y no existe sanitización, enviar ../../etc/passwd provoca la concatenación:

```
$$\text{Ruta Resultante} = \text{/var/www/images/} + \text{../../etc/passwd} \\ \text{implies /etc/passwd}$$
```

El kernel del sistema operativo normaliza la ruta:

1. /var/www/images/.. asciende a /var/www/
2. /var/www/.. asciende a /var/
3. /var/.. asciende a / (la raíz)
4. Por convención y diseño del núcleo en Linux, intentar subir de nivel desde la raíz (/..) mantiene al apuntador en /. Por lo tanto, cualquier secuencia redundante adicional de ../ continuará anclada en /, permitiendo ingresar directamente a /etc/ y leer passwd.

5.3. Análisis de Impacto en la Tríada CIA (Confidencialidad, Integridad y Disponibilidad)

- **Confidencialidad (Impacto Crítico):** Permite a un atacante no autenticado leer archivos arbitrarios del sistema, tales como /etc/passwd, credenciales de bases de datos embebidas en archivos de configuración (wp-config.php, .env), tokens de sesión, claves privadas SSH (/root/.ssh/id_rsa), código fuente y registros de auditoría (logs).
- **Integridad (Impacto Nulo a Alto, según el vector):** En vectores de solo lectura (como este laboratorio), la integridad del sistema no se altera directamente.
- **Disponibilidad (Impacto Medio a Potencial):** La lectura arbitraria de descriptores de archivos especiales o virtuales de Linux puede ocasionar el agotamiento de memoria, saturación del canal de red y bloqueos por denegación de servicio (DoS) en el hilo de atención del servidor web.

5.4. Variantes Principales de Path Traversal

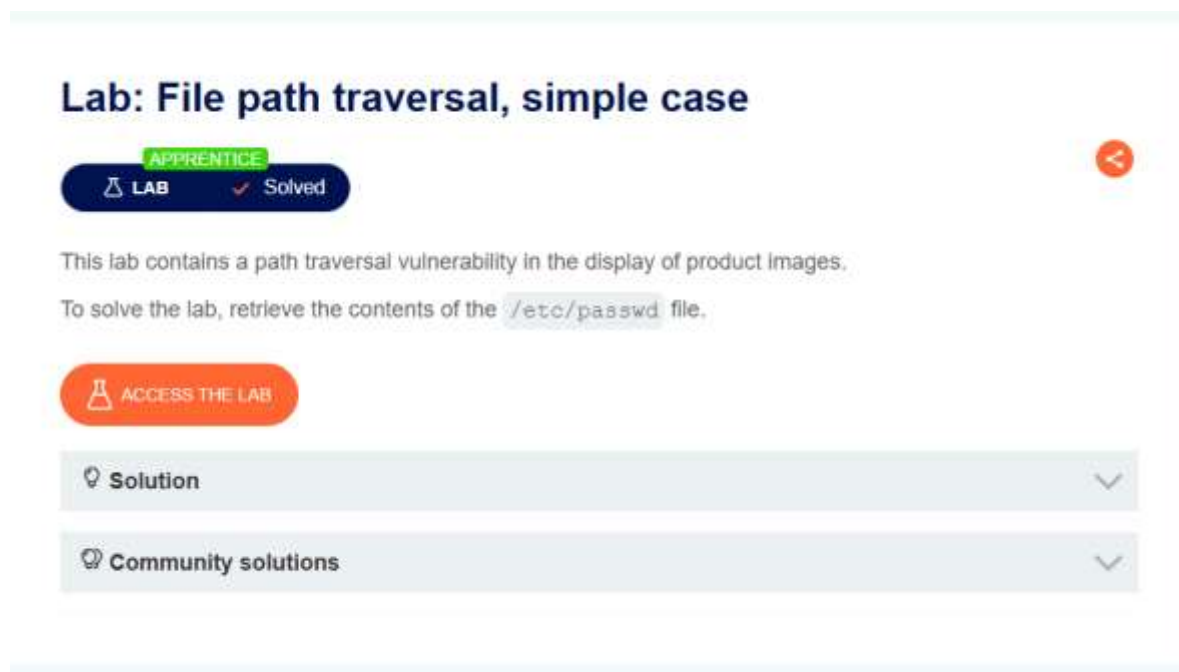
1. **Simple Case (Laboratorio actual):** No existe ningún filtro; el servidor pasa directamente la entrada a la función de apertura de archivos (open(), file_get_contents(), FileInputStream).
2. **Absolute Path Bypass:** La aplicación bloquea secuencias ../, pero acepta rutas que inician desde la raíz absoluta directa (/etc/passwd).

3. **Non-recursive Stripped Sequences:** El servidor aplica una sola pasada de filtrado eliminando la cadena ../. Puede evadirse anidando secuencias (....// se reduce a ../).
4. **Superfluous / Double URL Encoding:** Mecanismos de inspección periféricos (WAF) decodifican la URL una sola vez, pero el backend vuelve a procesarla, permitiendo el uso de payloads como %252e%252e%252f.
5. **Null Byte Injection (%00):** Utilizada en lenguajes como PHP pre-5.3.4 para truncar validaciones rígidas de extensión en cadenas tipo C (ej. ../../../../etc/passwd%00.png).

6. Procedimiento Paso a Paso y Uso Técnico de Burp Suite

Paso 1: Acceso e inicialización del entorno del laboratorio

Se ingresa al portal de PortSwigger Web Security Academy en la sección correspondiente al laboratorio *File path traversal, simple case*. Se identifica la descripción del problema y se hace clic sobre el botón naranja "**Access the lab**" para generar la instancia vulnerable en la nube.

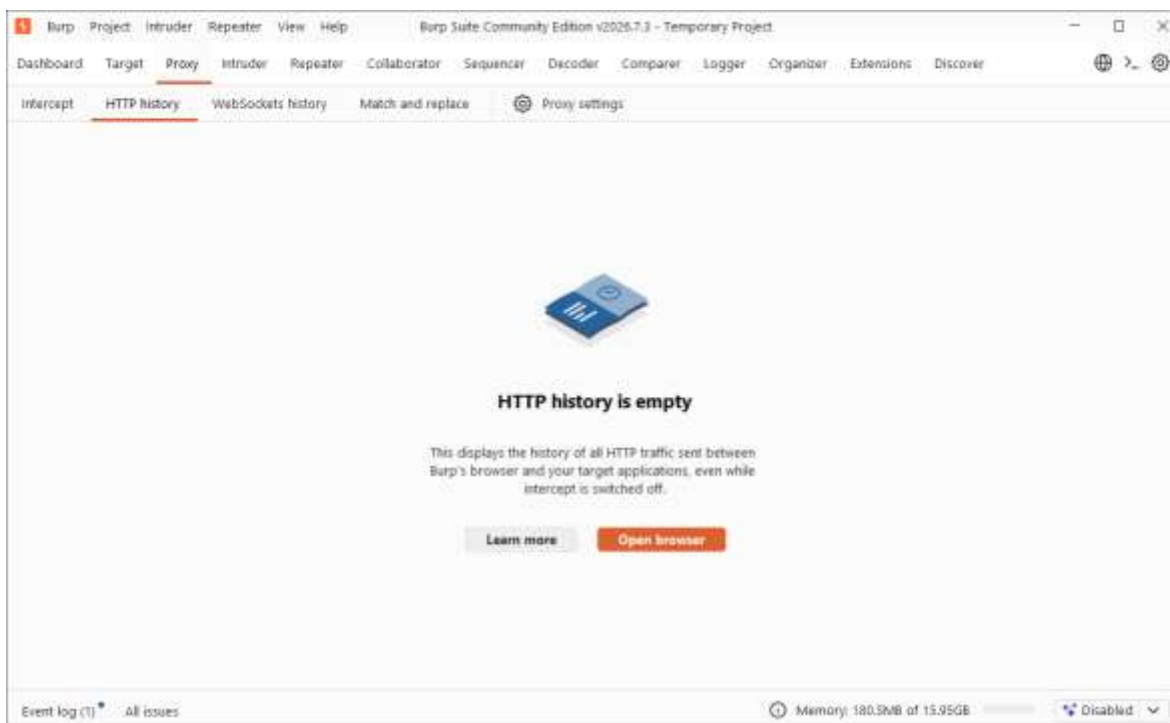


Evidencia 01: Página inicial del laboratorio en PortSwigger con el título, descripción y el botón interactivo "ACCESS THE LAB" destacado antes de iniciar la prueba.

Paso 2: Configuración del entorno de auditoría e inicio del navegador

Se abre Burp Suite Community Edition y se navega hasta la pestaña **Proxy > HTTP history**. Para garantizar que el tráfico fluya de forma controlada a través del puerto de intercepción (127.0.0.1:8080), se presiona el botón "**Open browser**", el cual lanza

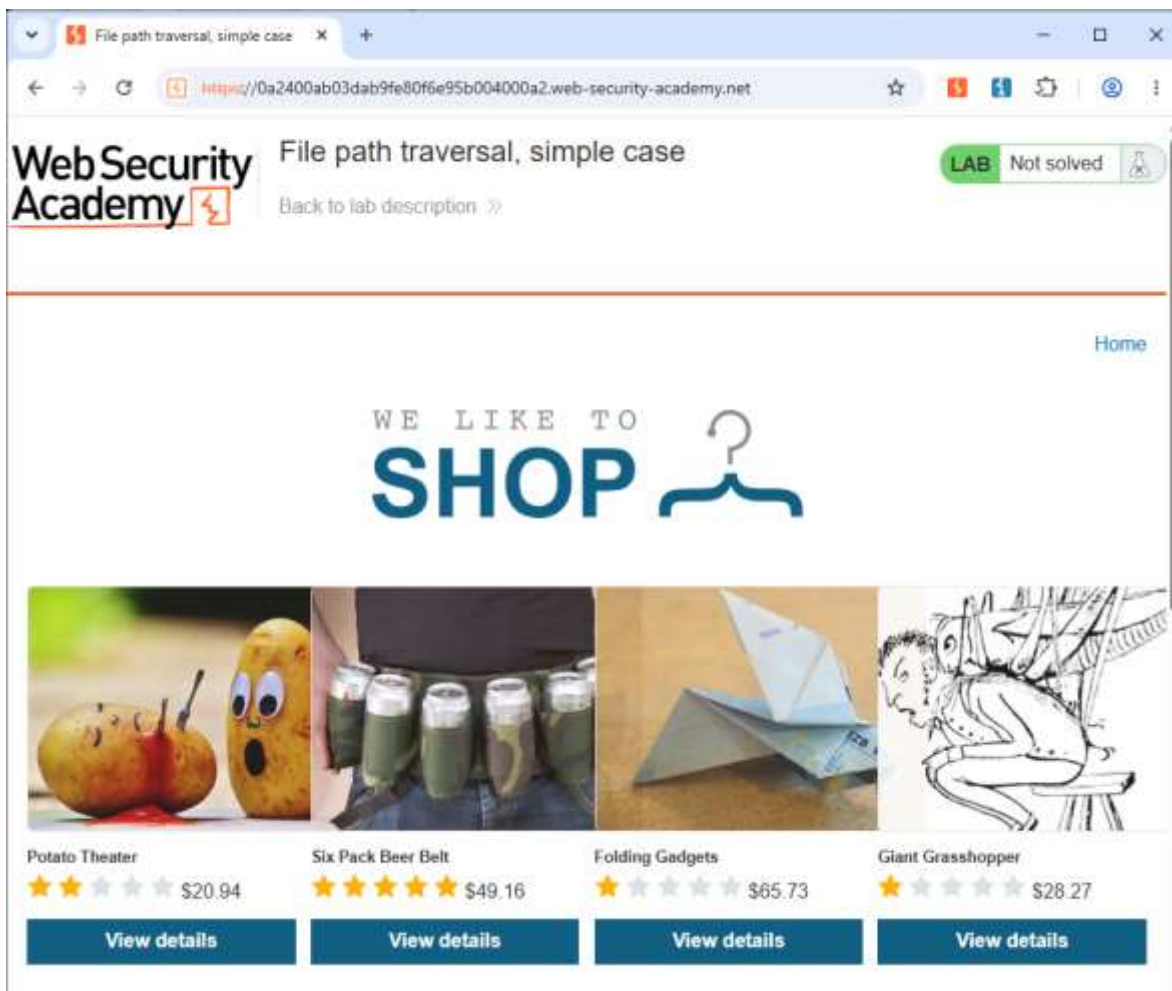
una instancia de Chromium preconfigurada que integra automáticamente los certificados CA de PortSwigger y Burp Suite, registrando cada paquete enviado y recibido.



Evidencia 02: Panel de Burp Suite en Proxy > HTTP history mostrando el historial limpio y el botón "Open browser" listo para desplegar el navegador integrado.

Paso 3: Carga y mapeo visual de la aplicación objetivo

En el navegador integrado de Burp Suite, se introduce la URL asignada a la instancia dinámica del laboratorio (ej. https://0a2400ab03dab9fe...web-security-academy.net/). Se visualiza la tienda en línea ("WE LIKE TO SHOP") con un catálogo de productos e imágenes estáticas, asegurando que el objetivo responde adecuadamente a nivel HTTP.



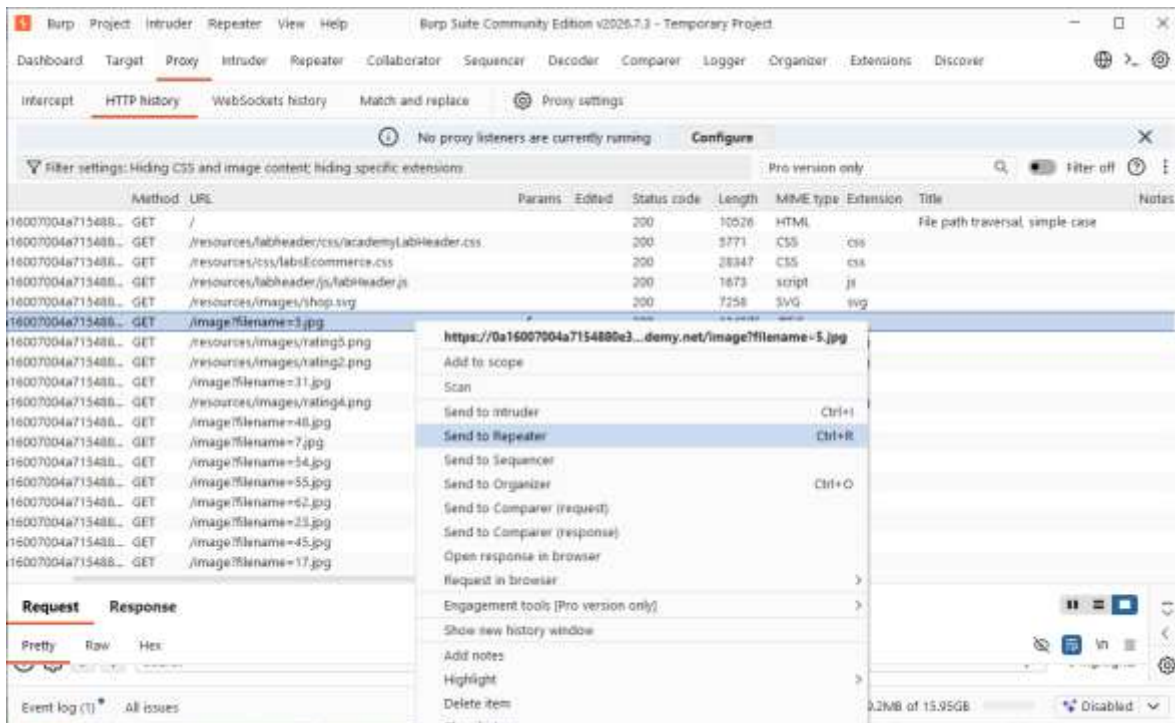
Evidencia 03: Vista del sitio web comercial "WE LIKE TO SHOP" renderizado dentro del navegador seguro de Burp Suite.

Paso 4: Inspección de peticiones HTTP y selección del vector en HTTP History

Al navegar en la página, se regresa a la pestaña **Proxy > HTTP history** en Burp Suite. Se localizan las solicitudes correspondientes a la carga de recursos multimedia. Se observa que los productos cargan imágenes mediante peticiones estructuradas como:

GET /image?filename=5.jpg HTTP/2

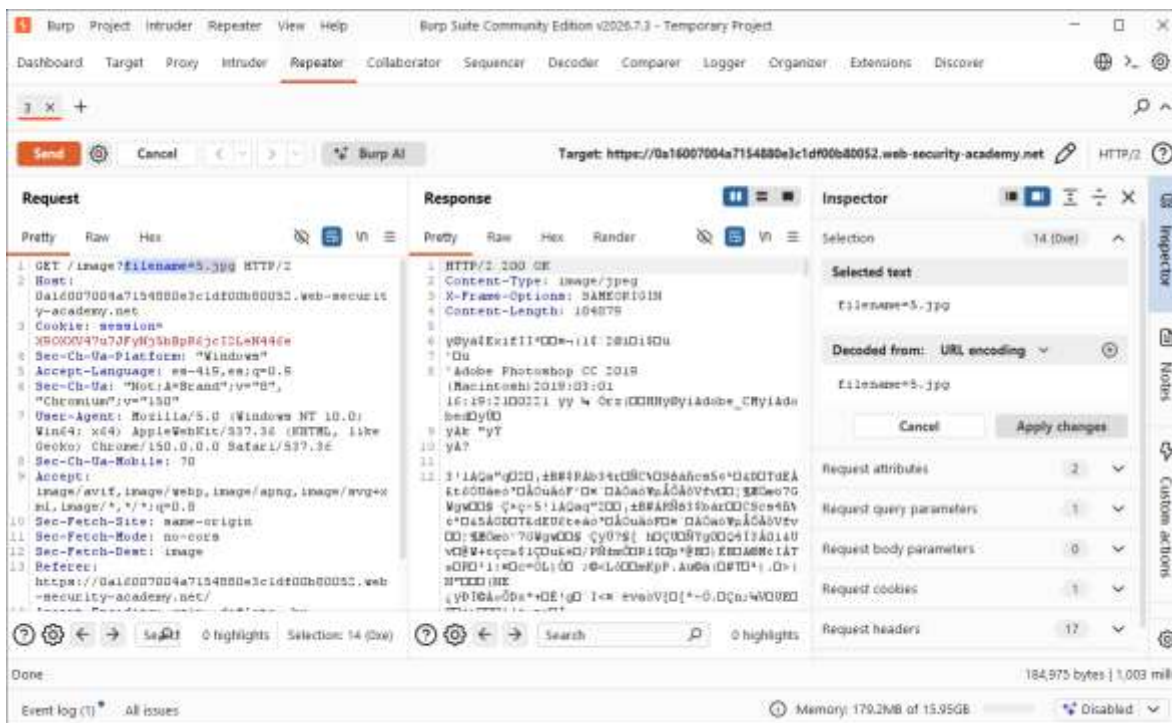
Se selecciona la solicitud con el botón secundario del ratón y se elige la opción **"Send to Repeater"** (o mediante la combinación de teclas Ctrl + R), lo que permite aislar y manipular la petición de forma controlada sin alterar la navegación general del explorador.



Evidencia 04: Historial HTTP en Burp Suite mostrando el listado de recursos solicitados y el menú contextual seleccionando la opción "Send to Repeater" sobre la petición /image?filename=5.jpg.

Paso 5: Aislamiento del parámetro vulnerable en el módulo Repeater

Dentro del entorno **Repeater**, se analiza la estructura integral de la petición original. En la sección *Request*, se localiza y sombrea el parámetro de consulta (*query parameter*) filename=5.jpg. El Inspector de Burp confirma que el valor actual corresponde a un archivo estático estándar de imagen y que el servidor carece de mecanismos defensivos en la cabecera para restringir la asignación manual de dicho parámetro.



Evidencia 05: Módulo Repeater con la petición base, mostrando el fragmento filename=5.jpg resaltado en el editor de texto y en el panel Inspector listo para ser sustituido.

Paso 6: Inyección del Payload y exfiltración de /etc/passwd

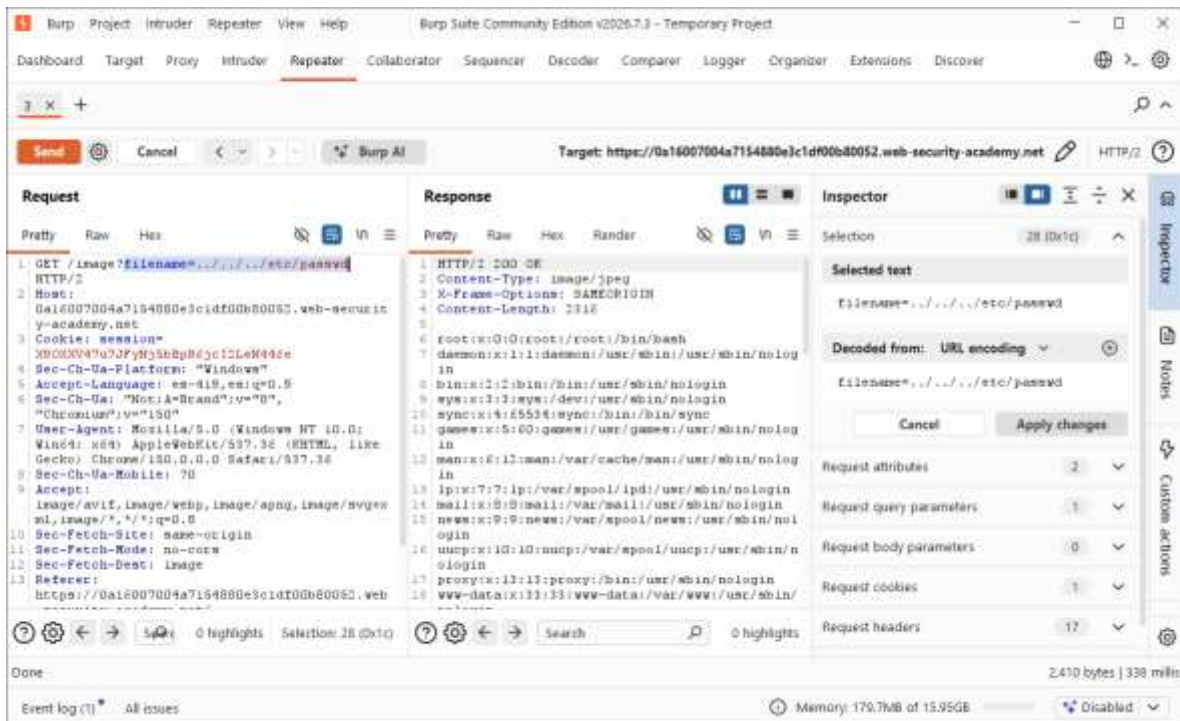
Se sustituye el nombre del archivo de imagen por el payload deDirectory Traversal:

../../../../etc/passwd

La petición queda formulada como:

GET /image?filename=../../../../etc/passwd HTTP/2

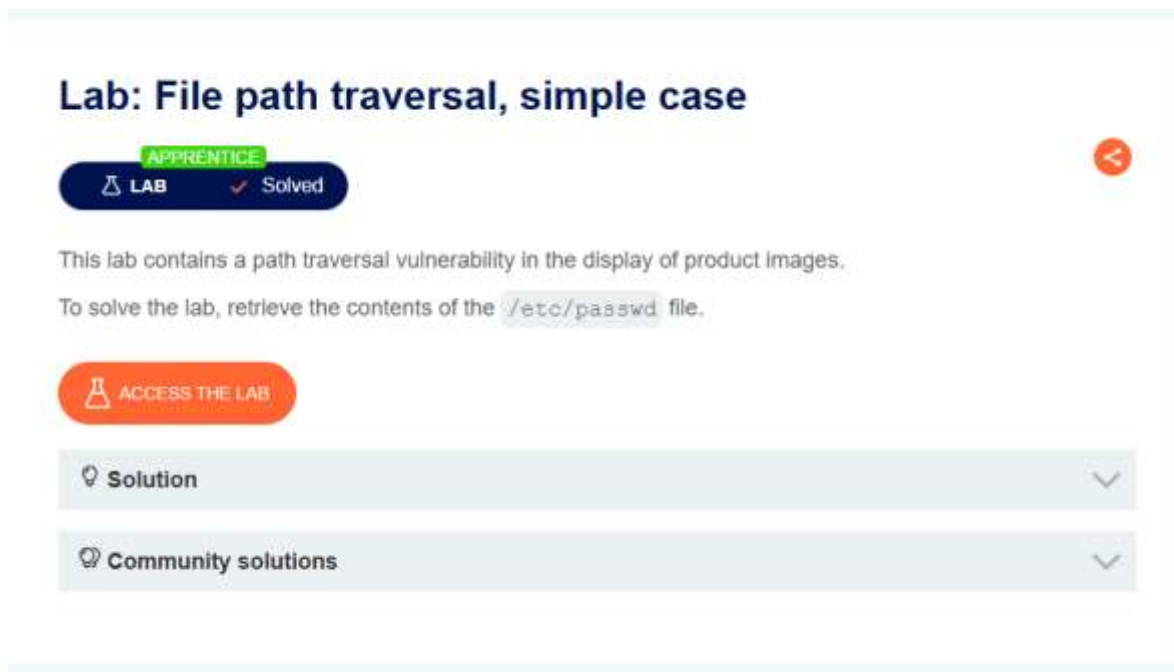
Se hace clic sobre el botón **Send**. En la sección **Response**, el servidor devuelve un código de estado HTTP/2 200 OK. En el cuerpo (*body*) de la respuesta no se recibe una imagen JPEG/PNG, sino el volcado completo en texto plano del archivo /etc/passwd del sistema operativo Linux que hospeda el servicio web, evidenciando cuentas como root:x:0:0:..., daemon, bin, sync y www-data.



Evidencia 06: Módulo Repeater tras presionar "Send", mostrando a la izquierda el payload inyectado `../../../../etc/passwd` y a la derecha la respuesta HTTP 200 con el volcado completo del archivo `/etc/passwd`.

Paso 7: Validación y confirmación del reto resuelto

Se regresa a la ventana del navegador web y se recarga la página principal del laboratorio. El banner superior de la aplicación actualiza su estado de "Not solved" a un letrero verde que confirma: "Solved" / "Lab Solved".



Evidencia 07: Pantalla del portal de PortSwigger con la etiqueta en verde confirmando la resolución satisfactoria: "Lab Solved".

7. Registro de Evidencias Técnicas

ID Evidencia	Elemento Requerido	Descripción Técnica
EVID-01	Inicialización del entorno	Interfaz de inicio de PortSwigger con la opción <i>Access the lab</i> .
EVID-02	Configuración del Proxy	Vista de <i>Proxy > HTTP history</i> y botón <i>Open browser</i> .
EVID-03	Aplicación objetivo	Interfaz web de la tienda "WE LIKE TO SHOP" bajo auditoría.
EVID-04	Interceptación de tráfico	Identificación de <code>GET /image?filename=...</code> y envío a Repeater (Ctrl + R).
EVID-05	Localización del vector	Identificación del parámetro vulnerable <code>filename</code> en la solicitud HTTP.
EVID-06	Inyección y Exfiltración	Petición modificada (<code>../../../../etc/passwd</code>) y respuesta HTTP 200 con el archivo.
EVID-07	Acreditación de resolución	Confirmación visual del estado "Lab Solved" en la plataforma.

8. Explicación Técnica del Payload

Payload Utilizado:

../../../../etc/passwd

Análisis de la Explotación:

1. **Punto de Inyección:** Parámetro filename en la ruta /image.
2. **Estructura del Path:** La aplicación almacena los recursos gráficos dentro de un subdirectorio estándar, usualmente /var/www/images/ o similar.
3. **Mapeo de la Secuencia:**
 - Primer ../: Sale del directorio images/ y se ubica en /var/www/.
 - Segundo ../: Sale de www/ y se ubica en /var/.
 - Tercer ../: Sale de var/ y alcanza el directorio raíz /.
 - etc/passwd: Concatena la ruta descendente hacia la carpeta de configuración del sistema (/etc/) y selecciona el archivo de cuentas de usuario (passwd).
4. **Por qué funciona:** La aplicación no realiza ninguna validación de tipo Whitelist ni invoca mecanismos de canonicalización. El sistema operativo lee directamente el archivo solicitado a través del descriptor del sistema de ficheros y el servidor web lo devuelve en el cuerpo de la respuesta HTTP, conservando las cabeceras originales pero transmitiendo texto legible en lugar de bytes de imagen.
5. **Estructura del contenido exfiltrado (/etc/passwd):** El archivo recuperado expone registros estructurados bajo el estándar POSIX:

```
$$\text{usuario}:\text{clave}:\text{UID}:\text{GID}:\text{GECOS}:\text{director  
io\_home}:\text{shell}$$$$
```

- root:x:0:0:root:/root:/bin/bash: Confirma al superusuario root con UID 0.
- www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin: Revela la cuenta de servicio bajo la cual opera el servidor web, confirmando sus permisos y el shell restringido (nologin).

9. Mitigación Recomendada y Principios de Defensa en Profundidad

Para remediar de forma definitiva la vulnerabilidad de File Path Traversal, se deben aplicar las siguientes medidas en la arquitectura de la aplicación:

9.1. Eliminar la Entrada Directa del Usuario en Operaciones de Ficheros

La estrategia más robusta consiste en **no utilizar nombres de archivo directos provenientes de solicitudes externas**. En su lugar, se recomienda:

- Emplear identificadores numéricos indirectos (por ejemplo, id=5 en base de datos) que asocien internamente el nombre real del archivo en un registro mapeado de forma estricta.
- Emplear una lista blanca estricta (*Allowlisting*): Si el usuario debe proveer un nombre, verificar que coincida exclusivamente con una lista predefinida de archivos autorizados o que el formato cumpla con una expresión regular rígida alfanumérica sin signos de puntuación:

`^[a-zA-Z0-9_-]+\.(jpg|jpeg|png)$`

9.2. Verificación de Rutas Mediante Canonicalización (Path Canonicalization)

Si la aplicación requiere interactuar con el sistema de archivos utilizando nombres provistos, debe resolverse la ruta canónica del recurso y validar que pertenezca inequívocamente al directorio base designado.

9.3. Principio de Menor Privilegio y Enjaulamiento (Hardening del Entorno)

- **Permisos del proceso web:** Asegurar que el usuario bajo el cual corre el servidor web (por ejemplo, `www-data` o `nobody`) tenga permisos mínimos de lectura restringidos exclusivamente al directorio de la aplicación, bloqueando la lectura de `/etc/` u otros directorios sensibles del sistema mediante permisos POSIX y ACLs.
- **Aislamiento del entorno:** Desplegar los servicios dentro de contenedores aislados (Docker/LXC) o entornos enjaulados (*chroot jail*), complementados con módulos de seguridad del kernel como **SELinux** o **AppArmor**, impidiendo que incluso ante un desbordamiento o evasión de ruta el servidor acceda al sistema de ficheros real del host.

10. Referencias bibliográficas

- **MITRE Corporation.** (2024). CWE-22: Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal'). Common Weakness Enumeration. <https://cwe.mitre.org/data/definitions/22.html>
- **Open Web Application Security Project.** (2021). A01:2021 – Broken Access Control. OWASP Top 10. https://owasp.org/Top10/A01_2021-Broken_Access_Control/
- **Open Web Application Security Project.** (2023). Path Traversal. OWASP Foundation. https://owasp.org/www-community/attacks/Path_Traversal
- **PortSwigger.** (s. f.). Directory traversal (File path traversal). PortSwigger Web Security Academy. Recuperado el 6 de septiembre de 2026, de <https://portswigger.net/web-security/file-path-traversal>

- **PortSwigger.** (s. f.). Lab: File path traversal, simple case. PortSwigger Web Security Academy. Recuperado el 6 de septiembre de 2026, de <https://portswigger.net/web-security/file-path-traversal/lab-simple>